LAPORAN PRAKTIKUM PERTEMUAN 3
“Form Input dan State Management
”



Disusun Oleh :
Fiery Ardiansyah (20240801092)

Dosen Pengampu :
JEFRY SUNUPURWA ASRI , S.Kom., M.Kom

Mata Kuliah :
Pemrograman Mobile
CR001

FAKULTAS ILMU KOMPUTER
UNIVERSITAS ESA UNGGUL
2026

Bagian A, B dan D: Input Dasar dengan TextField dan Form dengan Validasi dan Aplikasi daftar dengan Provider

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

void main() {
  runApp(
    ChangeNotifierProvider(
      create: (_) => TugasModel(),
      child: const MyApp(),
    ),
  );
}

// MODEL TUGAS

class Tugas {
  String judul;
  bool selesai;

  Tugas(this.judul, {this.selesai = false});
}

class TugasModel extends ChangeNotifier {
  final List<Tugas> _items = [];

  List<Tugas> get items => List.unmodifiable(_items);

  int get jumlahSelesai => _items.where((t) => t.selesai).length;

  void tambah(String judul) {
    _items.add(Tugas(judul));
    notifyListeners();
  }

  void toggle(int index) {
    _items[index].selesai = !_items[index].selesai;
    notifyListeners();
  }

  void hapus(int index) {
    _items.removeAt(index);
    notifyListeners();
  }
}

// MY APP

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Praktikum 3',
      theme: ThemeData(
        colorSchemeSeed: Colors.blue,
        useMaterial3: true,
      ),
      home: const HomePage(),
    );
  }
}

// MENU UTAMA
```

```
class HomePage extends StatelessWidget {
  const HomePage({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Praktikum 3'),
      ),
      body: ListView(
        padding: const EdgeInsets.all(16),
        children: [
          ElevatedButton(
            onPressed: () {
              Navigator.push(
                context,
                MaterialPageRoute(
                  builder: (_) => const InputPage(),
                ),
              );
            },
            child: const Text('Bagian A - Input Dasar'),
          ),
          const SizedBox(height: 12),
          ElevatedButton(
            onPressed: () {
              Navigator.push(
                context,
                MaterialPageRoute(
                  builder: (_) => const FormPage(),
                ),
              );
            },
            child: const Text('Bagian B - Form Pendaftaran'),
          ),
          const SizedBox(height: 12),
          ElevatedButton(
            onPressed: () {
              Navigator.push(
                context,
                MaterialPageRoute(
                  builder: (_) => const TugasPage(),
                ),
              );
            },
            child: const Text('Bagian D - Daftar Tugas'),
          ),
        ],
      ),
    );
  }
}

// BAGIAN A - INPUT DASAR

class InputPage extends StatefulWidget {
  const InputPage({super.key});

  @override
  State<InputPage> createState() => _InputPageState();
}

class _InputPageState extends State<InputPage> {
  final _controller = TextEditingController();
```

```
  String _salam = '';

  @override
  void dispose() {
    _controller.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Input Dasar'),
      ),
      body: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          children: [
            TextField(
              controller: _controller,
              decoration: const InputDecoration(
                labelText: 'Nama',
                border: OutlineInputBorder(),
              ),
            ),
            const SizedBox(height: 12),
            ElevatedButton(
              onPressed: () {
                setState(
                    () => _salam = 'Halo, ${_controller.text}!',
                );
              },
              child: const Text('Sapa'),
            ),
            const SizedBox(height: 12),
            Text(
              _salam,
              style: const TextStyle(fontSize: 20),
            ),
          ],
        ),
      ),
    );
  }
}

// BAGIAN B - FORM DENGAN VALIDASI

class FormPage extends StatefulWidget {
  const FormPage({super.key});

  @override
  State<FormPage> createState() => _FormPageState();
}

class _FormPageState extends State<FormPage> {
  final _formKey = GlobalKey<FormState>();

  final _nama = TextEditingController();
  final _email = TextEditingController();

  String? _jurusan;
  bool _setuju = false;

  @override
  void dispose() {
```

```
  _nama.dispose();
  _email.dispose();
  super.dispose();
}

void _kirim() {
  if (_formKey.currentState!.validate()) {
    final jurusan = _jurusan ?? '-';

    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(
        content: Text(
          'Terdaftar: ${_nama.text} ($jurusan)',
        ),
      ),
    );
  }
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text('Form Pendaftaran'),
    ),
    body: Form(
      key: _formKey,
      child: ListView(
        padding: const EdgeInsets.all(16),
        children: [
          TextFormField(
            controller: _nama,
            decoration: const InputDecoration(
              labelText: 'Nama lengkap',
              border: OutlineInputBorder(),
            ),
            validator: (v) =>
            (v == null || v.trim().isEmpty)
                ? 'Nama wajib diisi'
                : null,
          ),
          const SizedBox(height: 12),
          TextFormField(
            controller: _email,
            keyboardType: TextInputType.emailAddress,
            decoration: const InputDecoration(
              labelText: 'Email',
              border: OutlineInputBorder(),
            ),
            validator: (v) {
              if (v == null || !v.contains('@')) {
                return 'Email tidak valid';
              }

              return null;
            },
          ),
          const SizedBox(height: 12),
          DropdownButtonFormField<String>(
            decoration: const InputDecoration(
              labelText: 'Jurusan',
              border: OutlineInputBorder(),
            ),
            items: const [
              DropdownMenuItem(
                value: 'TI',
```

```
                child: Text('Teknik Informatika'),
              ),
              DropdownMenuItem(
                value: 'SI',
                child: Text('Sistem Informasi'),
              ),
              DropdownMenuItem(
                value: 'TE',
                child: Text('Teknik Elektro'),
              ),
            ],
            onChanged: (v) => setState(
                () => _jurusan = v,
            ),
            validator: (v) =>
            v == null ? 'Pilih jurusan' : null,
          ),
          CheckboxListTile(
            title: const Text(
              'Saya menyetujui ketentuan',
            ),
            value: _setuju,
            controlAffinity:
            ListTileControlAffinity.leading,
            onChanged: (v) => setState(
                () => _setuju = v ?? false,
            ),
          ),
          ElevatedButton(
            onPressed: _setuju ? _kirim : null,
            child: const Text('Daftar'),
          ),
        ],
      ),
    ),
  );
 }
}

// BAGIAN D - DAFTAR TUGAS

class TugasPage extends StatelessWidget {
 const TugasPage({super.key});

 @override
 Widget build(BuildContext context) {
  final model = context.watch<TugasModel>();

  return Scaffold(
    appBar: AppBar(
      title: Text(
        'Tugas (${model.jumlahSelesai}/${model.items.length})',
      ),
    ),
    body: ListView.builder(
      itemCount: model.items.length,
      itemBuilder: (context, i) {
        final t = model.items[i];

        return ListTile(
          leading: Checkbox(
            value: t.selesai,
            onChanged: (_) =>
                context.read<TugasModel>().toggle(i),
          ),
          title: Text(
```

```
            t.judul,
            style: TextStyle(
              decoration: t.selesai
                  ? TextDecoration.lineThrough
                  : null,
            ),
          ),
          trailing: IconButton(
            icon: const Icon(Icons.delete),
            onPressed: () =>
                context.read<TugasModel>().hapus(i),
          ),
        );
      },
    ),
    floatingActionButton: FloatingActionButton(
      onPressed: () {
        Navigator.push(
          context,
          MaterialPageRoute(
            builder: (_) => const TambahPage(),
          ),
        );
      },
      child: const Icon(Icons.add),
    ),
  );
 }
}

// BAGIAN D - TAMBAH TUGAS

class TambahPage extends StatefulWidget {
 const TambahPage({super.key});

 @override
 State<TambahPage> createState() => _TambahPageState();
}

class _TambahPageState extends State<TambahPage> {
 final _controller = TextEditingController();

 @override
 void dispose() {
  _controller.dispose();
  super.dispose();
 }

 void _simpan() {
  final judul = _controller.text.trim();

  if (judul.isEmpty) return;

  context.read<TugasModel>().tambah(judul);

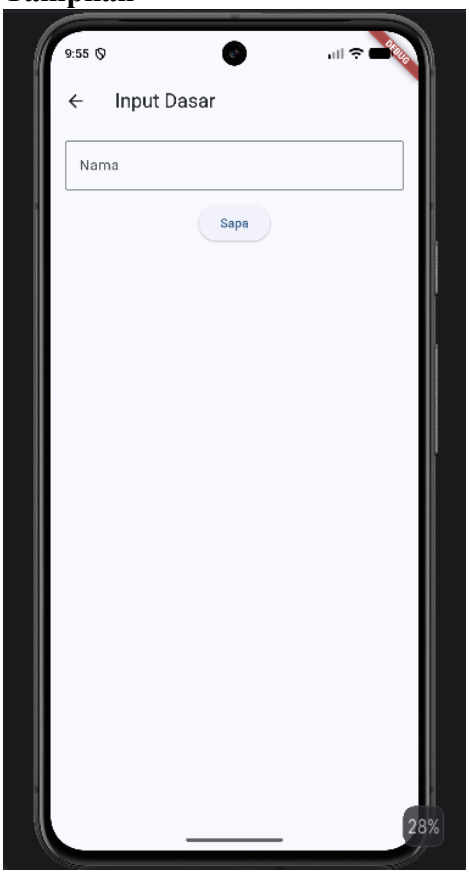
  Navigator.pop(context);
 }

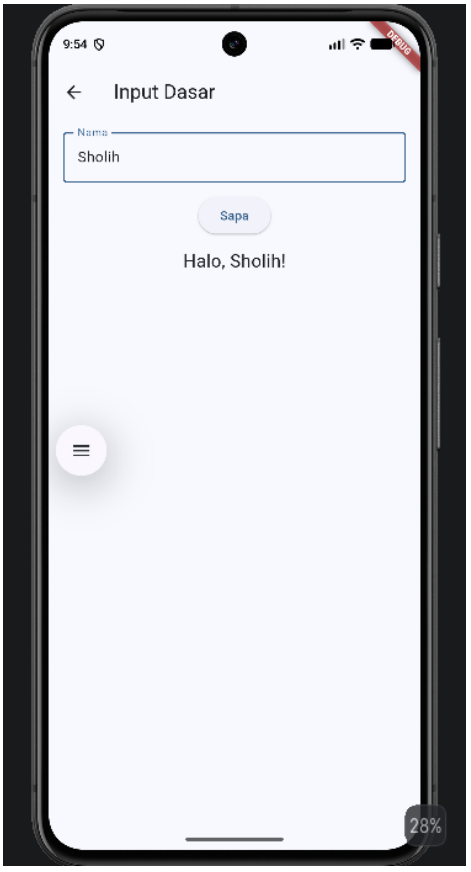
 @override
 Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text('Tambah Tugas'),
    ),
    body: Padding(
      padding: const EdgeInsets.all(16),
```

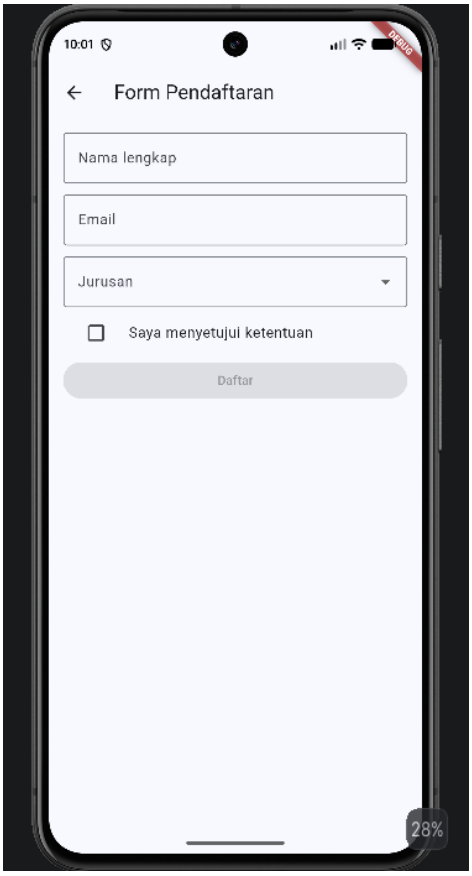
```
      child: Column(
        children: [
          TextField(
            controller: _controller,
            autofocus: true,
            decoration: const InputDecoration(
              labelText: 'Judul tugas',
              border: OutlineInputBorder(),
            ),
            onSubmitted: (_) => _simpan(),
          ),
          const SizedBox(height: 12),
          ElevatedButton(
            onPressed: _simpan,
            child: const Text('Simpan'),
          ),
        ],
      ),
    ),
  );
 }
}
```

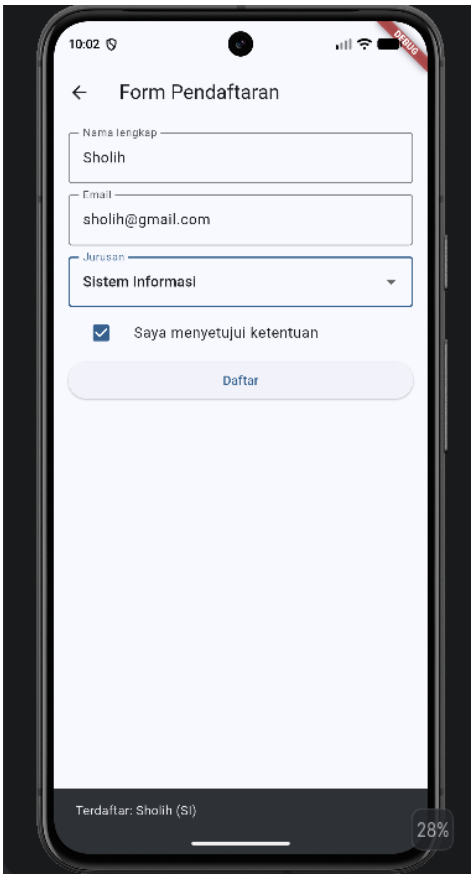
Syntax	Fungsi
TextEditingController	Untuk mengambil isi tulisan yang dimasukkan.
setState()	Untuk memperbarui tampilan setelah ada perubahan.
onChanged	Menjalankan perubahan saat isi tulisan berubah.
dispose()	Untuk melepaskan controller saat halaman tidak digunakan.
Form	Mengelompokkan beberapa isian dalam satu form.
GlobalKey<FormState>	Untuk mengontrol dan memeriksa form.
TextFormField	Kolom input yang bisa diberi pemeriksaan.
validator	Untuk menentukan aturan pemeriksaan isian.
DropdownButtonFormField	Membuat pilihan dalam bentuk dropdown.
Checkbox	Membuat pilihan yang dapat dicentang.

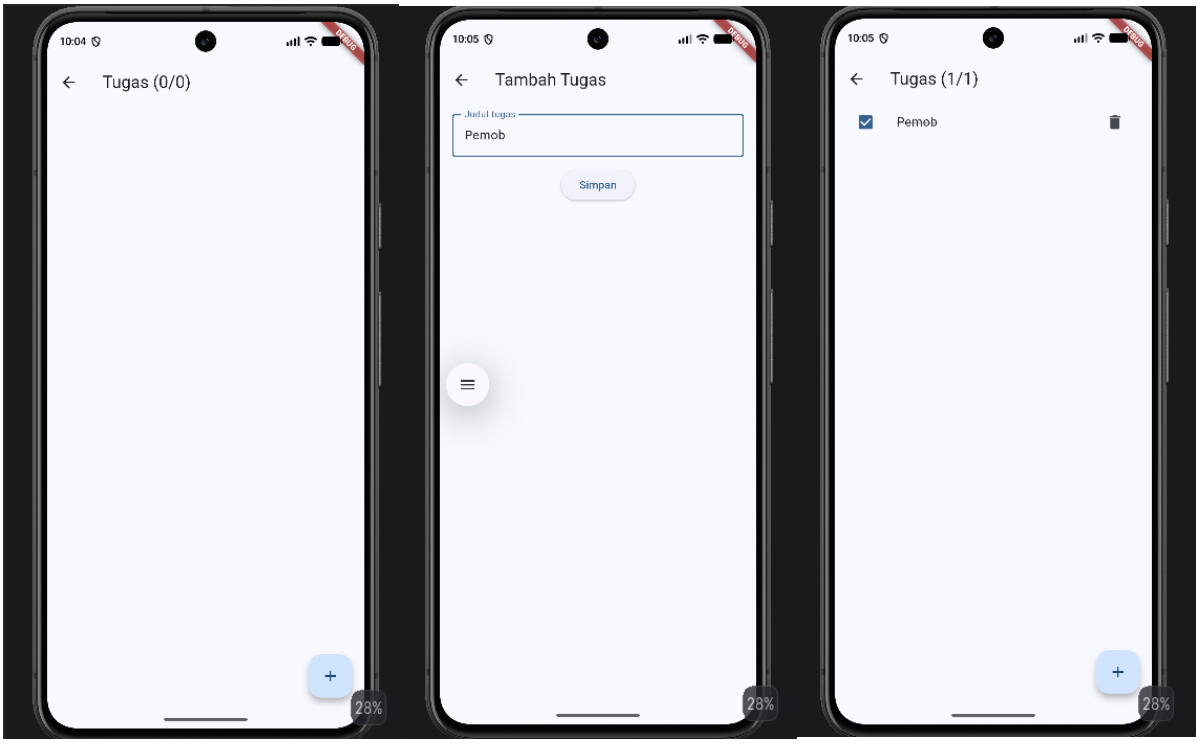
Tampilan


9:55
Input Dasar
Nama
Sapa
28%


9:54
Input Dasar
Nama
Sholih
Sapa
Halo, Sholih!
28%


10:01
Form Pendaftaran
Nama lengkap
Email
Jurusan
Saya menyetujui ketentuan
Daftar
28%


10:02
Form Pendaftaran
Nama lengkap
Sholih
Email
sholih@gmail.com
Jurusan
Sistem Informasi
Saya menyetujui ketentuan
Daftar
Terdaftar: Sholih (SI)
28%



Bagian C: Mengapa Butuh State Management?

Pada Bagian A dan B, state hidup di dalam satu halaman (setState). Bayangkan aplikasi daftar tugas: halaman daftar menampilkan tugas, sedangkan halaman tambah membuat tugas baru. Kedua halaman perlu berbagi data yang sama. Mengoper data lewat constructor dan callback dari satu halaman ke halaman lain cepat menjadi rumit. Solusinya: pindahkan data ke satu objek bersama yang dapat diakses kedua halaman. Inilah peran Provider.

Syntax	Fungsi
ChangeNotifier	Tempat menyimpan data yang bisa berubah.
ChangeNotifierProvider	Membagikan data agar bisa digunakan di beberapa halaman.
notifyListeners()	Memberi tahu bahwa data telah berubah.
context.watch()	Mengambil data dan mengikuti perubahannya.
context.read()	Mengambil data untuk melakukan suatu tindakan.
List.unmodifiable()	Membuat daftar yang tidak bisa diubah langsung.
where()	Memilih data berdasarkan kondisi tertentu.

Latihan Mandiri

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

class Tugas {
  String judul;
  bool selesai;

  Tugas(this.judul, {this.selesai = false});
}

class TugasModel extends ChangeNotifier {
  final List<Tugas> _items = [];

  List<Tugas> get items => List.unmodifiable(_items);

  int get jumlahSelesai =>
      _items.where((t) => t.selesai).length;

  void tambah(String judul) {
    _items.add(Tugas(judul));
    notifyListeners();
  }

  void toggle(int index) {
    _items[index].selesai = !_items[index].selesai;
    notifyListeners();
  }
```

```
  void hapus(int index) {
    _items.removeAt(index);
    notifyListeners();
  }

  void hapusSelesai() {
    _items.removeWhere((t) => t.selesai);
    notifyListeners();
  }
}

void main() {
  runApp(
    ChangeNotifierProvider(
      create: (_) => TugasModel(),
      child: const MyApp(),
    ),
  );
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Daftar Tugas',
      theme: ThemeData(
        colorSchemeSeed: Colors.blue,
        useMaterial3: true,
      ),
      home: const TugasPage(),
    );
  }
}

class TugasPage extends StatelessWidget {
  const TugasPage({super.key});

  @override
  Widget build(BuildContext context) {
    final model = context.watch<TugasModel>();

    return Scaffold(
      appBar: AppBar(
      title: Text(
          'Tugas (${model.jumlahSelesai}/${model.items.length})',
        ),
        actions: [
          IconButton(
            icon: const Icon(Icons.delete_sweep),
```

```
      tooltip: 'Hapus tugas selesai',
      onPressed: () {
        context.read<TugasModel>().hapusSelesai();
      },
    ),
  ],
),
body: model.items.isEmpty
    ? const Center(
  child: Text(
  'Belum ada tugas',
  style: TextStyle(
      fontSize: 18,
    ),
  ),
)
    : ListView.builder(
  itemCount: model.items.length,
  itemBuilder: (context, i) {
    final t = model.items[i];

    return ListTile(
      leading: Checkbox(
      value: t.selesai,
      onChanged: (_) {
          context.read<TugasModel>().toggle(i);
        },
      ),
      title: Text(
        t.judul,
        style: TextStyle(
          decoration: t.selesai
              ? TextDecoration.lineThrough
              : null,
        ),
      ),
      trailing: IconButton(
        icon: const Icon(Icons.delete),
        onPressed: () {
        context.read<TugasModel>().hapus(i);
        },
      ),
    );
  },
),
floatingActionButton: FloatingActionButton(
  onPressed: () async {
    final hasil = await Navigator.push(
      context,
      MaterialPageRoute(
        builder: (_) => const TambahPage(),
      ),
```

```
        );

        if (hasil == true && context.mounted) {
          ScaffoldMessenger.of(context).showSnackBar(
          const SnackBar(
              content: Text('Tugas ditambahkan'),
            ),
          );
        }
      },
      child: const Icon(Icons.add),
    ),
  );
 }
}

class TambahPage extends StatefulWidget {
 const TambahPage({super.key});

 @override
 State<TambahPage> createState() => _TambahPageState();
}

class _TambahPageState extends State<TambahPage> {
 final _formKey = GlobalKey<FormState>();
 final _controller = TextEditingController();

 @override
 void dispose() {
  _controller.dispose();
  super.dispose();
 }

 void _simpan() {
  if (_formKey.currentState!.validate()) {
   final judul = _controller.text.trim();

   context.read<TugasModel>().tambah(judul);

   Navigator.pop(context, true);
  }
 }

 @override
 Widget build(BuildContext context) {
  return Scaffold(
   appBar: AppBar(
    title: const Text('Tambah Tugas'),
   ),
   body: Padding(
    padding: const EdgeInsets.all(16),
    child: Form(
```

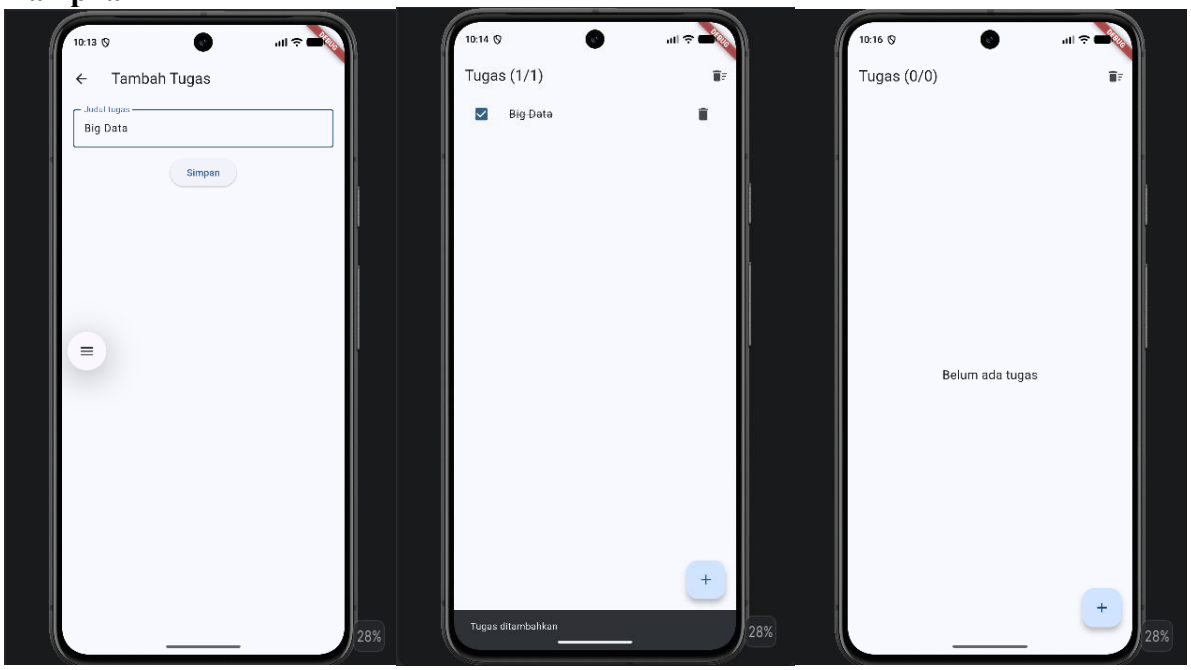
```
        key: _formKey,
        child:  Column(
        children: [
           TextFormField(
             controller: _controller,
             autofocus: true,
             decoration: const InputDecoration(
               labelText: 'Judul tugas',
               border: OutlineInputBorder(),
             ),
             validator: (value) {
               if (value == null ||
                   value.trim().isEmpty) {
                 return 'Judul tugas wajib diisi';
               }

               if (value.trim().length < 3) {
                 return 'Judul tugas minimal 3 karakter';
               }

               return null;
             },
             onFieldSubmitted: (_) => _simpan(),
           ),
           const SizedBox(height: 12),
           ElevatedButton(
             onPressed: _simpan,
             child: const Text('Simpan'),
           ),
         ],
       ),
     ),
   ),
  );
 }
}
```

Syntax	Fungsi
removeWhere()	Menghapus semua data yang memenuhi kondisi.
trim()	Menghilangkan spasi di awal dan akhir tulisan.
length	Menghitung jumlah karakter.
Navigator.push()	Membuka halaman baru.
Navigator.pop()	Kembali ke halaman sebelumnya.
ScaffoldMessenger	Menampilkan pesan singkat seperti SnackBar.
isEmpty	Memeriksa apakah daftar masih kosong.

Tampilan



Tugas Mandiri

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

class Barang {
  String nama;
  int jumlah;
  String kategori;
  bool sudahDibeli;

  Barang({
    required this.nama,
    required this.jumlah,
    required this.kategori,
    this.sudahDibeli = false,
  });
}

class BelanjaModel extends ChangeNotifier {
  final List<Barang> _items = [];

  List<Barang> get items => List.unmodifiable(_items);

  int get jumlahBelumDibeli =>
      _items.where((barang) => !barang.sudahDibeli).length;
```

```
  void tambahBarang(
      String nama,
      int jumlah,
      String kategori,
      ) {
    _items.add(
      Barang(
      nama: nama,
        jumlah: jumlah,
        kategori: kategori,
      ),
    );

    notifyListeners();
  }

  void toggleDibeli(int index) {
    _items[index].sudahDibeli =
    !_items[index].sudahDibeli;

    notifyListeners();
  }

  void hapusBarang(int index) {
    _items.removeAt(index);

    notifyListeners();
  }
}

void main() {
  runApp(
    ChangeNotifierProvider(
      create: (_) => BelanjaModel(),
      child: const MyApp(),
    ),
  );
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Daftar Belanja',
      theme: ThemeData(
        colorSchemeSeed: Colors.green,
        useMaterial3: true,
      ),
      home: const DaftarPage(),
```

```
    );
  }
}

class DaftarPage extends StatelessWidget {
  const DaftarPage({super.key});

  @override
  Widget build(BuildContext context) {
    final model = context.watch<BelanjaModel>();

    return Scaffold(
      appBar: AppBar(
      title: Text(
          'Daftar Belanja (${model.jumlahBelumDibeli})',
        ),
      ),
      body: model.items.isEmpty
          ? const Center(
        child: Text(
          'Belum ada barang',
          style: TextStyle(
          fontSize: 18,
          ),
        ),
      )
          : ListView.builder(
        itemCount: model.items.length,
        itemBuilder: (context, index) {
          final barang = model.items[index];

          return ListTile(
            leading: Checkbox(
              value: barang.sudahDibeli,
              onChanged: (_) {
                context
                    .read<BelanjaModel>()
                    .toggleDibeli(index);
              },
            ),
            title: Text(
              barang.nama,
              style: TextStyle(
                decoration: barang.sudahDibeli
                    ? TextDecoration.lineThrough
                    : null,
              ),
            ),
            subtitle: Text(
              '${barang.jumlah} - ${barang.kategori}',
            ),
            trailing: IconButton(
```

```
            icon: const Icon(Icons.delete),
            onPressed: () {
              context
                  .read<BelanjaModel>()
                  .hapusBarang(index);
            },
          ),
        );
      },
    ),
    floatingActionButton: FloatingActionButton(
      onPressed: () {
        Navigator.push(
          context,
          MaterialPageRoute(
            builder: (_) => const TambahBarangPage(),
          ),
        );
      },
      child: const Icon(Icons.add),
    ),
  );
 }
}

class TambahBarangPage extends StatefulWidget {
 const TambahBarangPage({super.key});

 @override
 State<TambahBarangPage> createState() =>
     _TambahBarangPageState();
}

class _TambahBarangPageState
   extends State<TambahBarangPage> {
 final _formKey = GlobalKey<FormState>();

 final _namaController = TextEditingController();
 final _jumlahController = TextEditingController();

 String? _kategori;

 final List<String> _kategoriList = [
  'Makanan',
  'Minuman',
  'Kebutuhan Rumah',
  'Kebutuhan Pribadi',
 ];

 @override
 void dispose() {
  _namaController.dispose();
```

```
  _jumlahController.dispose();
  super.dispose();
}

void _simpan() {
  if (_formKey.currentState!.validate()) {
    final nama = _namaController.text.trim();
    final jumlah = int.parse(
      _jumlahController.text.trim(),
    );

    context.read<BelanjaModel>().tambahBarang(
      nama,
      jumlah,
      _kategori!,
    );

    Navigator.pop(context);
  }
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text('Tambah Barang'),
    ),
    body: Padding(
      padding: const EdgeInsets.all(16),
      child: Form(
        key: _formKey,
        child:  Column(
        children: [
            TextFormField(
              controller: _namaController,
              decoration: const InputDecoration(
              labelText: 'Nama barang',
              hintText: 'Contoh: Beras',
                border: OutlineInputBorder(),
              ),
              validator: (value) {
                if (value == null ||
                    value.trim().isEmpty) {
                  return 'Nama barang wajib diisi';
                }

                return null;
              },
            ),

            const SizedBox(height: 16),
```

```
TextFormField(
  controller: _jumlahController,
  keyboardType: TextInputType.number,
  decoration: const InputDecoration(
  labelText: 'Jumlah',
    hintText: 'Contoh: 2',
    border: OutlineInputBorder(),
  ),
  validator: (value) {
    if (value == null ||
        value.trim().isEmpty) {
      return 'Jumlah wajib diisi';
    }

    final jumlah =
    int.tryParse(value.trim());

    if (jumlah == null) {
      return 'Jumlah harus berupa angka';
    }

    if (jumlah <= 0) {
      return 'Jumlah harus lebih dari 0';
    }

    return null;
  },
),

const SizedBox(height: 16),

DropdownButtonFormField<String>(
  value: _kategori,
  decoration: const InputDecoration(
    labelText: 'Kategori',
    border: OutlineInputBorder(),
  ),
  items: _kategoriList.map((kategori) {
    return DropdownMenuItem(
      value: kategori,
      child: Text(kategori),
    );
  }).toList(),
  onChanged: (value) {
  setState(() {
      _kategori = value;
    });
  },
  validator: (value) {
    if (value == null || value.isEmpty) {
      return 'Kategori wajib dipilih';
    }
```

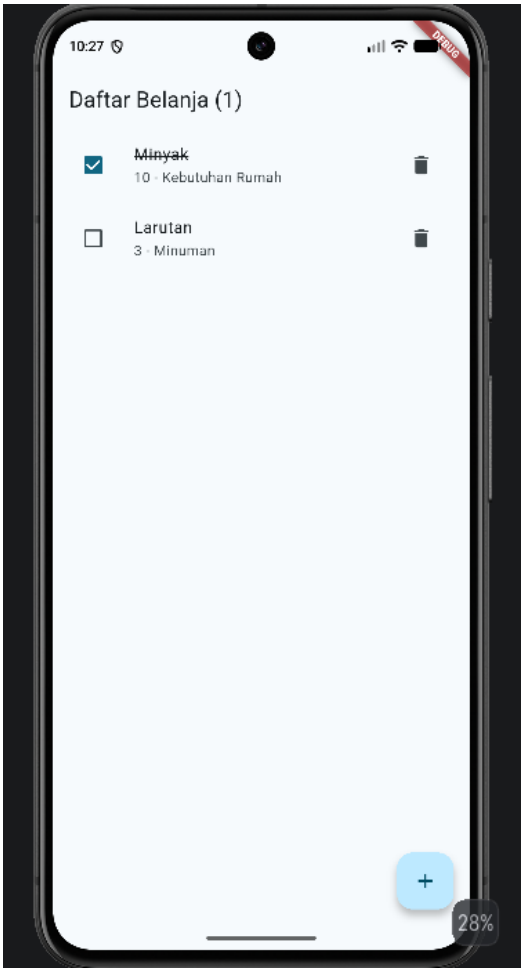
```
              return null;
            },
          ),

          const SizedBox(height: 24),

          SizedBox(
            width:  double.infinity,
            child: ElevatedButton(
            onPressed: _simpan,
              child: const Text('Simpan'),
            ),
          ),
        ],
      ),
    ),
  ),
 );
 }
}
```

Tampilan





Syntax	Fungsi
required	Menandakan data wajib diberikan.
int.tryParse()	Mengubah tulisan menjadi angka.
TextInputType.number	Menampilkan keyboard angka.
List<String>	Membuat daftar yang berisi beberapa teks.
!	Menandakan nilai dianggap tidak kosong atau membalik kondisi.

Pertanyaan Refleksi

1. **Mengapa TextEditingController harus di-dispose?**
Agar resource/memori yang digunakan controller dilepas dan tidak menyebabkan memory leak.

2. **Kapan cukup memakai setState, dan kapan sebaiknya beralih ke Provider?**
setState cukup untuk state yang hanya digunakan dalam satu halaman. Provider digunakan jika data perlu dipakai atau dibagikan oleh beberapa halaman/widget.

3. **Apa yang terjadi bila notifyListeners() lupa dipanggil? Mengapa?**
Perubahan data tidak langsung tampil di UI karena widget yang menggunakan Provider tidak diberi tahu bahwa data telah berubah.

4. **Mengapa di dalam onPressed kita memakai context.read, bukan context.watch?**
Karena onPressed hanya perlu mengambil data atau menjalankan fungsi tanpa perlu memantau perubahan data. context.watch digunakan ketika UI perlu otomatis diperbarui saat data berubah.